



Silas Reuel da Silva - Guia Para um Novo Usuário Ubuntu
Bauru-SP - 2026

O que é um diretório?.....	3
Comandos Básicos no Terminal.....	3
Obtendo o manual de comandos.....	3
Limpar o terminal.....	4
O histórico.....	4
Acessar pastas/diretórios.....	4
Criar pastas/diretórios.....	5
Criar arquivos.....	6
Ler conteúdo de arquivos.....	6
Listar conteúdo de uma pasta (arquivos, pastas).....	6
Remover/Apagar.....	6
Remover/Apagar arquivo.....	7
Remover/Apagar pasta.....	7
Entendendo o Sudo.....	7
Sudo su.....	8
Atualizando Pacotes do Sistema.....	8
O update.....	8
O upgrade.....	8
O autoremove.....	8
Instalando Softwares.....	9
O apt install.....	9
Editores de Texto no Terminal.....	9
Nano.....	9
Vim.....	11
Atualizando Aplicativos pelo Terminal.....	13
Criando Aliases (Apelidos).....	13
Personalizando o Terminal.....	14
Terminal Integrado VSCode.....	15

ATENÇÃO!!! Isso NÃO é um manual! Este arquivo contém apenas informações básicas para consulta rápida ou introdução ao sistema operacional.

Importante ressaltar que tudo aqui descrito foi testado em uma máquina física ou virtual a fim de averiguar a eficácia e inibir erros de informação.

O que é um diretório?

Diretórios são usados como sinônimo de pastas no computador, ou seja, a pasta *home* é um diretório, porém, o diretório refere-se ao conceito técnico e local de armazenamento.

Comandos Básicos no Terminal

Comandos são formados pela estrutura básica:

comando -flag argumentos

Comando → Programa ou ferramenta que se deseja executar

Flag (ou opções) → Modificam o comportamento do comando

Argumento → Dados adicionais que o comando precisa para funcionar

Exemplo:

Comando para remover diretório de forma recursiva

rm -rf pasta

OBS: Alguns comandos não necessitam de argumentos (**pwd**, **ls**).

A *Flag* não é obrigatória, sendo usada apenas em casos de necessidade.

Obtendo o manual de comandos

O comando a seguir entrega o manual/documentação do comando que será passado como argumento, por exemplo, manual do **ls**:

man ls

Importante declarar que a maioria dos comandos, exceto scripts personalizados, de terceiros ou ferramentas muito simples, podem ser consultados com o `man` ou usando a flag `--help`

`ls --help`

Limpar o terminal

Serve para o caso de ter muitas informações e você quer limpar o terminal para iniciar um novo processo

`clear`

O histórico

O terminal salva suas consultas, ou seja, cada comando fica salvo, e você pode listá-lo, estará na ordem de execução e numerado, o histórico salva todos os comandos então você terá comandos repetidos

`history`

Localizar posição atual no terminal

O comando a seguir vai mostrar o endereço de pastas onde você está, por exemplo: `/home/nome-usuario/Documentos`

Comando: `pwd`

Acessar pastas/diretórios

O comando abaixo acessa um diretório chamado `endereco`, em seguida acessa outro com nome `do_diretorio` e finalmente chega no diretório `Alvo`. Cada diretório é separado por `/` (barra simples). Pode ser que nos diretórios tivessem outros que não foram acessados nesse caso.

`cd endereco/do_diretorio/Alvo`

Este `../` significa que estou voltando um diretório, para voltar para o diretório antes de acessar o diretório *endereco* seria `../../../`
`cd ../`

A pasta *home* é representada pelo símbolo `~`, sendo assim, caso queira retornar de vários diretórios para *home* use o comando acima
`cd ~`

Caso queira voltar para um diretório que tenha acessado anteriormente sem ter que digitar a rota para esse diretório, use
`cd -`

Exemplo: Acessou o diretório de Documentos (`cd Documentos`), depois retornou e acessou Downloads (`cd ../Downloads`) e deseja retornar para o diretório anterior (Documentos), basta usar `cd -`

Criar pastas/diretórios

O comando abaixo cria uma pasta chamada *diretorio*
`mkdir diretorio`

O comando a seguir cria uma pasta chamada teste dentro da pasta *diretorio*, porém a pasta 'pai' já deve existir. Imagine que é o caso de estar em *home*, dentro há a pasta *diretorio*, porém você precisa criar *teste* ali dentro, mas não quer navegar para dentro de *diretorio* para criá-la, use o comando abaixo
`mkdir diretorio/teste`

O comando abaixo cria a pasta 'pai', neste caso *diretorio* e em seguida a pasta filha *teste*. Este é o caso de, por exemplo, estar em *home* e precisar criar uma pasta dentro de outra que ainda não existe, isso funciona graças à flag `-p` cujo significado é *parents*.
`mkdir -p diretorio/teste`

Criar arquivos

Existem alguns métodos, mas o `touch` irá apenas criar o arquivo

```
touch nome_do_arquivo
```

Ler conteúdo de arquivos

Este comando irá escrever no terminal todo o conteúdo do arquivo

```
cat nome_do_arquivo
```

Listar conteúdo de uma pasta (arquivos, pastas)

Lista arquivos e pastas visíveis dentro da pasta atual

```
ls
```

Lista todos os arquivos e pastas na pasta atual, incluindo os ocultos

```
ls -a
```

Lista todos os arquivos e pastas na pasta atual no formato de lista longa (proprietário, permissões, data de edição, etc)

```
ls -l
```

Como são as duas flags juntas, o comando lista todos os arquivos e pastas na pasta atual, no formato de lista longa (proprietário, permissões, data de edição, etc), inclui os ocultos

```
ls -la
```

Remover/Apagar

Para isso usa-se o comando `rm`, porém depende do que você vai remover

Remover/Apagar arquivo

Esse comando excluirá o arquivo mencionado no argumento

```
rm nome_do_arquivo
```

Remover/Apagar pasta

Comando para remover um diretório. **ATENÇÃO!** Ele falhará caso haja arquivos dentro da pasta

```
rmdir diretorio
```

Com a flag **-r** é possível usar *rm* para remover um diretório. **ATENÇÃO!** Ele removerá mesmo que haja arquivos na pasta

```
rm -r diretorio
```

Ou o comando

```
rm -rf diretorio
```

Com a flag **-f** ele não perguntará nada e irá ignorar arquivos e argumentos inexistentes (**--force**)

Entendendo o Sudo

SuperUser Do (superusuário faz , traduzido para português)

O comando **sudo** nada mais é do que a “palavrinha mágica” para executar tarefas administrativas no sistema, isso é, se seu usuário tem permissão de administrador, não adianta saber a palavra e não poder usar, talvez você deva comprar um cafézinho pro administrador, vai que ele te dá essa permissão...

Basicamente o root e outro usuário com acesso de administrador podem usar isso. Ao executar o comando, a senha de usuário será requisitada.

Normalmente você vai rodar

```
sudo comando
```

Sudo su

```
sudo su
```

Abre uma janela como root, dando as permissões de superusuário para o usuário que requisitou, sem a necessidade de usar sudo toda vez, para sair basta executar `exit`

Mas é preferível usar `sudo` antes do comando do que executar `sudo su`, já que comandos perigosos como `rm -rf /` podem ser executados sem aviso e causar danos ao sistema, podendo ocorrer principalmente com novos usuários.

Atualizando Pacotes do Sistema

Importante manter pacotes atualizados pela segurança e bom funcionamento destes.

O update

Esse comando procura nos servidores configurados em `/etc/apt/sources.list.d` e baixa os índices mais recentes dos pacotes instalados

```
sudo apt update
```

O upgrade

Esse comando compara as versões instaladas com as listas baixadas do `upgrade`, baixa e instala as atualizações, instalando novos pacotes necessários (dependências), mas não remove pacotes instalados

```
sudo apt upgrade
```

O autoremove

Esse comando remove pacotes e bibliotecas instalados automaticamente como dependências, mas que atualmente não são necessários por nenhum programa. Ele libera espaço em disco e limpa o sistema sendo ideal o uso periódico para manter o sistema enxuto

```
sudo apt autoremove
```

Instalando Softwares

Por vezes será necessário instalar alguns softwares via terminal, como Vim, Git e muitos outros

O apt install

Esse comando busca os pacotes, instala e configura no sistema de forma automatizada

```
sudo apt install software_desejado
```

Por exemplo, a instalação do editor de texto, Vim:

```
sudo apt install vim
```

Editores de Texto no Terminal

Essas ferramentas servem para alterar arquivos de texto como *.txt*, *.py*, *.rb*

Temos um editor que vem instalado por padrão que é o Nano, outro bem comum mas que deve ser instalado é o Vim

Nano

Criar arquivos

Para criar um arquivo vazio com nano você usará o comando

```
nano arquivo
```

Nesse momento você terá uma tela como essa:

```
GNU nano 7.2      teste

^G Ajuda      ^O Gravar      ^W Onde está?  ^K Recortar    ^T Executar    ^C Local
^X Sair        ^R Ler o arq   ^\ Substituir  ^U Colar       ^J Justificar  ^/ Ir p/ linha
```

Para salvá-lo use Ctrl+O, se você apenas sair, por estar vazio, o arquivo não será salvo, quando fizer isso ele pedirá um nome para salvar:

```
GNU nano 7.2      teste

Nome do arquivo para salvar: teste
^G Ajuda      M-D Formato DOS  M-A Anexar      M-B Arquivo reserva
^C Cancelar    M-M Formato Mac  M-P Pré-anexar  ^T Navegar
```

Você pode manter o nome ou trocar neste momento, se optar por manter o nome apenas pressione enter, caso tenha mudado ele pedirá para confirmar a troca de nome:

```
GNU nano 7.2      teste

Salvar o arquivo com um NOME DIFERENTE?
S Sim
N Não      ^C Cancelar
```

Apenas digite S, ele voltará para o modo de edição, basta sair da ferramenta.

Para sair da ferramenta edição de arquivo use Ctrl+X

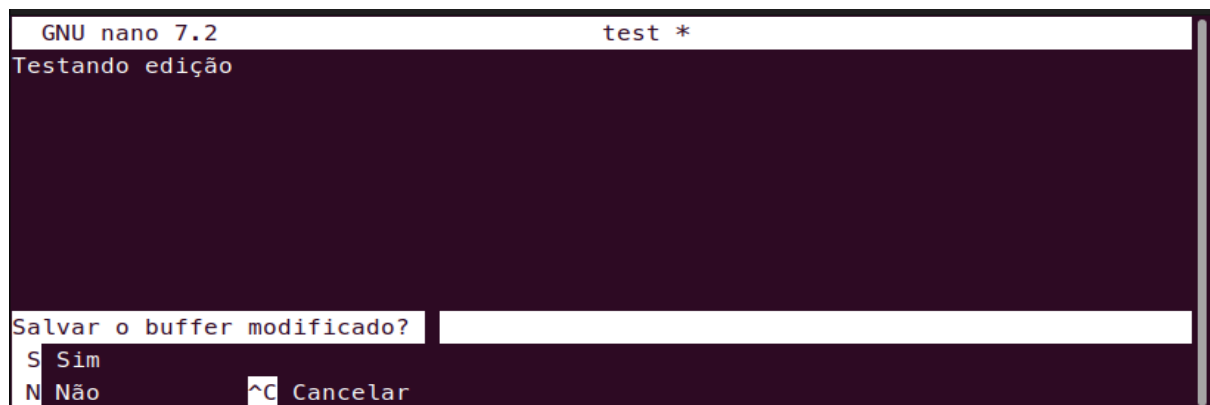
OBS: A ferramenta sempre diz o que fazer, quando houver um acento circunflexo (^) significa que deve usar Ctrl+ a letra que aparece após o acento.

Editando arquivos existentes

Abra o arquivo que deseja editar com o nano

```
nano arquivo_existente
```

Edite o arquivo com o que desejar, em seguida use Ctrl+O para gravar e o nano questionará se quer trocar o nome do arquivo. Se você não usar Ctrl+O e tentar sair, o nano vai perguntar se quer salvar o arquivo



Após salvar basta sair da ferramenta de edição

Vim

Vim também é um editor de texto porém não é nativo (instalado por padrão) como o Nano

Instalando o Vim

```
sudo apt install vim
```

Criar arquivos

Para criar arquivos basta rodar a ferramenta com o nome do arquivo que você deseja criar

```
vim arquivo
```

A ferramenta irá abrir com o arquivo em branco, para salvá-lo sem nenhuma informação e sair da ferramenta digite `:wq`

Caso você digite apenas `:q` ele sairá sem salvar pois está vazio e o novo arquivo será perdido

Criar arquivos

Basta você executar a ferramenta com o nome do arquivo que deseja editar

`vim arquivo`

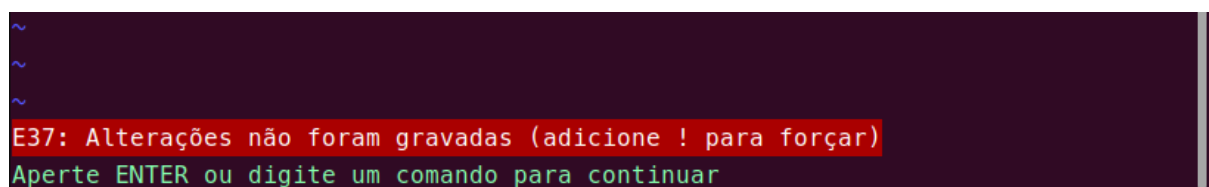
Ele irá abrir o arquivo, mas não no modo de edição, apenas visualização, você deve pressionar a tecla `i` para editar



```
teste
~
~
~
~
~
~
~
~
~
~
-- INSERÇÃO --          1,6          Tudo
```

Para salvar e sair use a tecla Esc, em seguida escreva `:wq`

Caso digite apenas `:q`, será lançado um alerta



```
~
~
~
E37: Alterações não foram gravadas (adicione ! para forçar)
Aperte ENTER ou digite um comando para continuar
```

Pressionando a tecla enter voltará para edição, digitando um comando ele executará (Ex: `:wq` irá salvar e sair)

Escrevendo `:q!` a saída será forçada e a edição será perdida, se digitar apenas `:q` o mesmo alerta será lançado, ao digitar apenas `:!` o vim encerrará mas mostrará o seguinte:

```
[Alterações não gravadas]
```

```
Aperte ENTER ou digite um comando para continuar
```

Permitindo que ao pressionar enter você volte a edição com os dados que adicionou ou simplesmente digite o comando para salvar

OBS: Todo comando vim é antecedido pelo símbolo de dois pontos (:)

Atualizando Aplicativos pelo Terminal

NÃO É NECESSÁRIO desinstalar o aplicativo.

Primeiramente deve baixar o arquivo de extensão .deb, em seguida mova para pasta onde ele está salvo, provavelmente Downloads:

```
cd Downloads
```

Instale a atualização:

```
sudo dpkg -i arquivo.deb
```

No caso de erro de dependências execute:

```
sudo apt install -f
```

Isso irá corrigir problemas de dependência automaticamente

Criando Aliases (Apelidos)

Aliases podem ser considerados atalhos para comandos, tornando mais fácil a rotina de trabalho ao se escrever pouco obtendo o mesmo resultado.

Tanto no Bash quanto no Zsh é a mesma forma, só muda o arquivo de configuração que será acessado, além de que o bash usará o oh my posh ao invés de oh my zsh:

.bashrc → Bash

.zshrc → Zsh

Acessar o arquivo com um editor de texto como nano ou vim:

```
vim ~/.zshrc → Para zsh
```

`vim ~/.bashrc` → Para bash

Criar o alias ao final do arquivo:

```
alias apelido="comando"
```

Exemplo:

```
alias gst="git status"
```

Personalizando o Terminal

Instalar zsh:

```
sudo apt install zsh -y
```

Instalar oh my zsh:

```
sh -c "$(curl -fsSL  
https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)"
```

Definir zsh como padrão:

```
chsh -s $(which zsh)
```

Reiniciar a máquina

Modificar para o tema desejado:

```
vim ~/.zshrc
```

Procurar pela linha ZSH_THEME e modificar para o tema desejado, logo em seguida salve o arquivo

POWER LEVEL 10K

Caso tenha instalado o powerLevel10k, digite no terminal zsh

```
p10k configure
```

Siga os passos de configuração

Terminal Integrado VSCode

No VSCode para usar o terminal integrado com as configuração do zsh atuais, acesse **settings.json**

Abrir configurações: **Ctrl + ,**

Em seguida pressione o botão **Edit as JSON**

Adicione ou edite o JSON, **não remova nada que já exista no arquivo**

```
"terminal.integrated.defaultProfile.linux": "zsh",  
"editor.fontFamily": "MesloLGS Nerd Font",  
"terminal.integrated.fontFamily": "MesloLGS Nerd  
Font Mono"
```

Ou use outras fontes compatíveis com Nerd Fonts

Caso queira usar comandos bash sem ter que remover o zsh do padrão, digite no terminal zsh

```
bash
```

Para retornar ao zsh digite no terminal

```
exit
```

Remover oh my zsh:

```
uninstall_oh_my_zsh
```

Caso o comando acima não funcione:

```
source ~/.oh-my-zsh/tools/uninstall.sh
```